Formal Verification in Hybrid Quantum–Classical Computing Systems: A Two-Stage Contract Framework for VQE

***Abstract*—The variational quantum eigensolver (VQE) couples classical decisions to uncertain quantum measurement estimates. We organize its verification around a shared execution model and a system-level contract library. Logical, numeric, and probabilistic contracts retain their own formalisms: NuXmv checks a bounded controller, Dafny proves real and integer kernels under assumptions, and Storm checks a Gaussian discrete-time Markov chain. A frozen pre-execution assurance record links these results to runtime evidence from the same canonical instance. In the final matched experiment, 400 finite-shot executions satisfy the observed logical and numeric obligations, with the prospective scope of termination and measurement accounting stated separately. The Gaussian model predicts 92.80 % success by 50 updates; 366 executions succeed, giving 91.50 % with a 95% Wilson interval of [88.36 %, 93.85 %]. Conditional update and shot costs are statistically compatible after adjustment for the supporting comparisons; an earlier marginal-interval disagreement was a statistical false alarm. Controlled faults distinguish observed violations, changed assumptions, and incomplete evidence. Retained layer-specific studies quantify model growth and runtime cost under their own configurations. The results support contract-wise cross-stage consistency on a two-qubit benchmark, while keeping model checking, mathematical proof, observed conformance, and statistical evidence within their respective scopes.**

***Index Terms*—VQE, formal verification, runtime verification, probabilistic model checking, contract traceability.**

I. Introduction

VQE estimates a Hamiltonian expectation using a parameterized quantum state and adjusts the parameters with a classical optimizer [1], [2]. Correct interaction requires more than a plausible final energy. Measurements must belong to the current candidate; aggregation must include the declared observables and coefficients; the optimizer must consume the values actually validated; and stopping must respect the declared policy. Finite-shot uncertainty introduces a different question: how often does the search reach its target within the resource bound?

These obligations call for different verification objects. A controller model captures ordering but abstracts arithmetic and measurement outcomes. A numeric proof establishes a calculation under input assumptions. A probabilistic model describes uncertain selections. Runtime evidence determines whether corresponding obligations hold in instrumented executions. Treating their results as interchangeable would obscure both the guarantees and the remaining assumptions.

The methodological principle is:

> The contract library defines what the VQE implementation is expected to satisfy. Pre-execution verification establishes these contracts prospectively, while runtime verification determines whether the corresponding obligations hold for concrete executions.

The principle does not imply that every obligation is fully proved at each stage. Contract matching must record the actual prospective subclaim, its assumptions, and the strength of the runtime counterpart.

We make three contributions. First, we define a two-stage architecture with a canonical instance, a family-owned contract library, and a frozen assurance record. Second, we explain how logical, numeric, and probabilistic obligations connect to runtime evidence without forcing a common formalism. Third, we evaluate those connections contract-wise using the final audited execution archive, controlled faults, and retained layer-specific cost studies. The paper asks: (RQ1) how are obligations organized around the shared execution model; (RQ2) what does each implemented layer establish; (RQ3) do the matched prospective and runtime results agree within their scopes; and (RQ4) what costs and fault behaviors are observed? RQ3 concerns consistency of evidence, rather than a proved theorem composing all tools.

Section II introduces the execution model and contract library. Section III gives each layer's technical meaning and procedure. Section IV evaluates the layers under their corresponding setups. Section V then evaluates their relationships collectively using the matched experiment.

II. Shared Execution Model and Contract Library

A. Execution and Contract Semantics

The shared execution view is a labeled transition system whose modes are initialization, preparation, measurement, aggregation, validation, optimizer commitment, stopping checks, and terminal closure. Its state retains a run identifier, candidate and evaluation identities, parameters, available measurement groups, validation readiness, update count, and resource accounting. Concrete identifiers strengthen the abstract controller's counters and epoch flags; their correspondence is an implementation mapping, not a machine-checked refinement theorem.

For a Pauli decomposition $H = \sum_i c_i P_i$, VQE evaluates

$$E(\theta) = \langle \psi(\theta) | H | \psi(\theta) \rangle, \qquad \widehat{E}(\theta) = \sum_i c_i \hat{e}_i. \tag{1}$$

Each contract records a family, obligation, pre-execution check, runtime counterpart, assumptions, and comparison rule. The implemented library is `config/contracts.json`. It documents these links; it is not a generator translating one specification into all three formalisms. Logical and numeric contracts apply at run level. Probabilistic contracts apply to repeated executions of a declared process.

Figure 1 shows the two stages. Pre-execution verification produces models, proof logs, and quantitative predictions before the designated execution begins. The assurance record freezes configuration, source hashes, tool identities, and artifact hashes. Stage 2 refuses execution when this evidence no longer matches. Runtime observations then support contract matching and verdict agreement. Changing a model or implementation requires a new assurance record and output directory; this feedback is manual.

Table I gives the contract mapping. A numeric-to-runtime match is established by interpreting concrete counts and optimizer inputs as the mathematical inputs of the kernel contract, checking its preconditions, and comparing the observed output to the declared arithmetic. It does not require the Python implementation to be verified by Dafny. For probabilistic matching, the runtime counterpart is an estimate over repeated executions, rather than an assertion that every run succeeds.

B. Canonical Instance and Frozen Boundary

The matched instance uses the following two-qubit benchmark operator, written in $q_1 \otimes q_0$ order:

$$H = 0.398\,YZ - 0.398\,ZI - 0.0113\,ZZ + 0.181\,XX. \quad (2)$$

The repository retains legacy "h2" names, but the operator has no documented molecular mapping and is not presented as a molecular-hydrogen Hamiltonian. Its reference minimum eigenvalue is $E_{\mathrm{ref}} = -0.702930394$ in the operator's numerical units. We use the same operator across the matched projections, without assigning chemical-accuracy meaning to the target.

The circuit applies H to both qubits, the first pair of RZ rotations, Y to both, $\mathrm{CX}(0 \to 1)$, H to both, the second pair of RZ rotations, and Y to both. Four integer indices z give angles $\theta_j = 2\pi z_j/16$, with wrap-around. The optimizer cycles through coordinates; each update remeasures the current, minus-one, and plus-one candidates and chooses the smallest integer energy score. Ties prefer current, then minus, then plus. The integer coefficients are the declared decimal coefficients scaled by 10^4.

Table II fixes the boundary. The run stops on first ideal-energy success or normal budget exhaustion; an online contract problem gives an error closure. The success oracle uses exact simulation of the selected point. It is a benchmarking predicate available to this simulator, not a hardware-observed stopping certificate or an energy-stability policy. There are 8 successful grid points. Each failed run remains in the probability denominator and consumes its budget.

III. Technical Meaning and Procedure of Each Layer

A. Logical Verification

NuXmv [3] checks a finite controller with measurement-group counters, candidate readiness, an update counter, and epoch flags. Measurement and environment outcomes are nondeterministic; numeric energies and quantum states are abstracted. Representative LTL obligations are

$$\begin{aligned} &\mathbf{G}(\mathrm{aggregate} \Rightarrow \mathrm{groups} = 3),\\ &\mathbf{G}(\mathrm{select} \vee \mathrm{commit} \Rightarrow \mathrm{ready} = 3),\\ &\mathbf{G}(\mathrm{commit} \Rightarrow \mathrm{energyEpoch} = \mathrm{epoch}). \end{aligned} \quad (3)$$

Readiness resets at the next update; terminal states are absorbing. The procedure generates the controller from the frozen bound, runs NuXmv, and stores every returned property and verdict. The result is exhaustive for the encoded model.

L4 requires special care. The final property set checks terminal absorption, that a budget-terminal state has the bound counter, and that commits occur below the bound. It does not establish that success and budget exhaustion must take their intended terminal branches. The audit found that changing either branch to abort leaves these properties true. Consequently, the paper reports only the verified subclaim prospectively. Runtime reason checks are additional evidence. The controller's presence of separate terminal branches does not itself supply the missing property.

B. Numeric Verification and Its Runtime Match

Dafny [4] proves kernels with mathematical real and integer semantics. A normalization routine assumes a positive shot count and an admissible event count. Weighted-sum routines assume matching term sequences and expectations in $[-1, 1]$, and establish

$$-\sum_i |c_i| \le \sum_i c_i \hat{e}_i \le \sum_i |c_i|. \quad (4)$$

Additional kernels accumulate natural-number shot records, preserve the grid under modular wrap, and choose the minimum integer score with the declared tie priority. Verification logs are frozen with the source modules.

Runtime matching uses raw measurements rather than assuming a reported expectation is correct. For outcome counts $n_{g,o}$ in group g, let $s_{i,o} \in \{-1, +1\}$ be the declared Pauli parity. The reference values are

$$\hat{e}_i^{\mathrm{ref}} = \frac{\sum_o s_{i,o} n_{g,o}}{\sum_o n_{g,o}}, \qquad E^{\mathrm{ref}} = \sum_i c_i \hat{e}_i^{\mathrm{ref}}. \quad (5)$$

The checker reconstructs these as exact rational numbers and compares the concrete floating-point expectations, contributions, and energy to them within the frozen tolerance. It also checks the integer scores and the actual values recorded as consumed by the optimizer. Thus the link is a shared mathematical obligation instantiated on observed inputs and outputs. There is no proved compiler refinement from the Dafny kernels to Python floating-point operations.

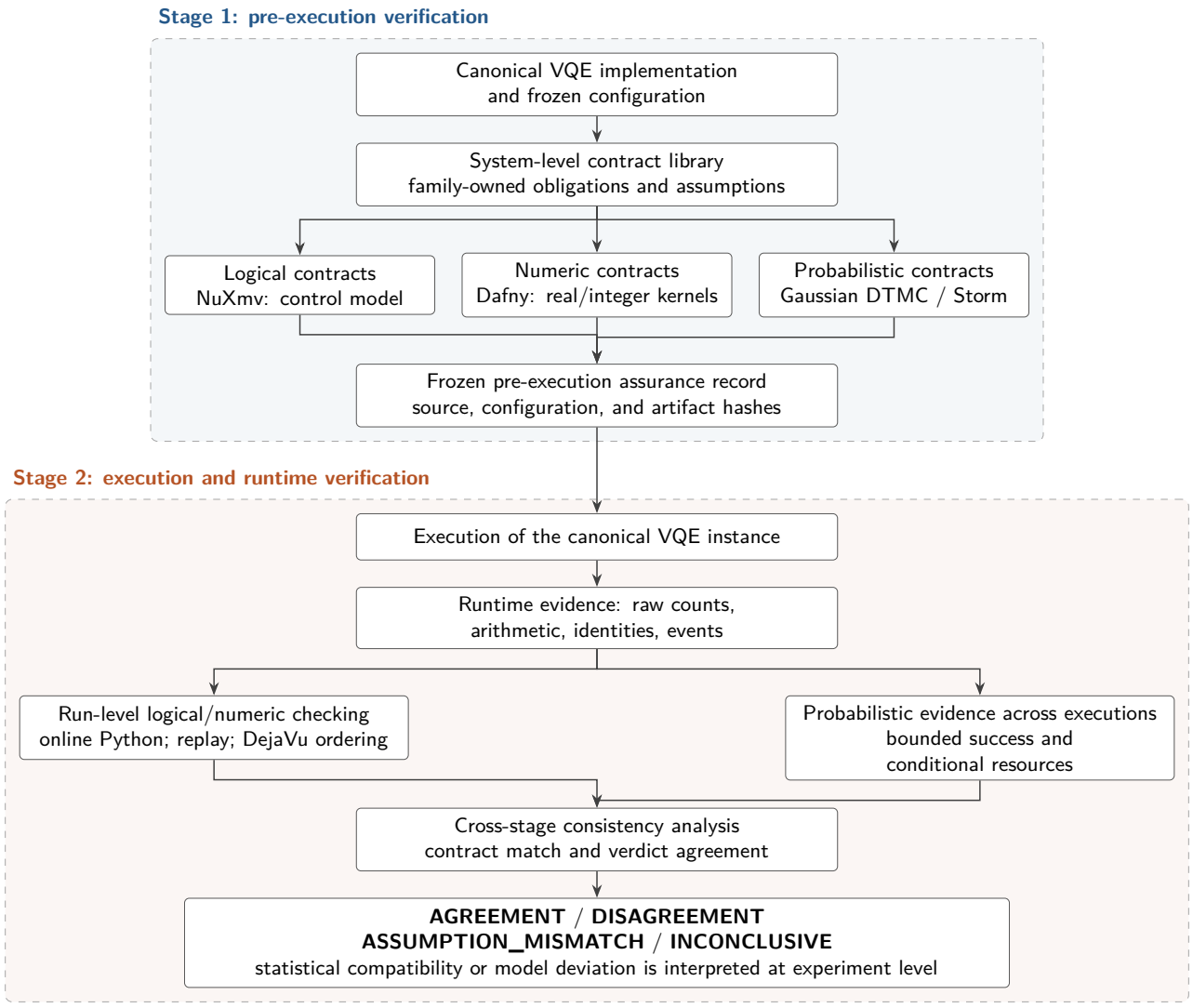


Fig. 1. Two-stage verification architecture for the canonical VQE instance. The contract families retain heterogeneous verification objects. The frozen record precedes execution; runtime checking combines run-level logical/numeric evidence with probabilistic evidence across repeated executions. The displayed verdict names match the implementation. A probabilistic disagreement denotes a model discrepancy only when assumptions and the statistical decision rule justify that interpretation. The architecture supplies traceability rather than an equivalence among NuXmv, Dafny, and Storm.

The boundaries matter. The normalization lemma uses the single-event count form; multi-qubit parity interpretation is trusted. The shot-accumulation proof establishes a list sum; it does not prove that list entries are unique physical executions. Runtime identities and completeness checks discharge these observational conditions under faithful instrumentation. Missing inputs yield incomplete evidence; changed shot allocations invalidate the probabilistic configuration. Proving consequences of a valid-record predicate does not prove that an arbitrary record is valid.

C. Probabilistic Verification

For each candidate, the Gaussian abstraction matches the ideal mean energy and the shot-estimator variance. Terms measured in the same group retain their covariance:

$$\sigma^2(z) = \frac{1}{N_g} \sum_g \mathrm{Var}_{\mathrm{Born}} \left(\sum_{i \in g} c_i s_i \right), \tag{6}$$

where all groups receive $N_g = 1,024$ shots. Groups, candidate measurements, and repeated executions are independent; terms within a group need not be. Approximating a candidate energy by $X_j \sim \mathcal{N}(\mu_j, \sigma_j^2)$ gives its selection probability

$$\Pr(j = \operatorname*{argmin}_\ell X_\ell) = \int f_j(x) \prod_{\ell \neq j} [1 - F_\ell(x)]\, dx. \tag{7}$$

The implementation evaluates this integral by configured quadrature and a normal-CDF approximation, then normalizes rows. The continuous model gives ties probability zero; discrete finite-shot execution applies the explicit tie rule.

TABLE I

SYSTEM-LEVEL CONTRACT TRACEABILITY. EACH ROW CONNECTS A PROSPECTIVE OBJECT TO CONCRETE EVIDENCE AND ITS INTERPRETATION. ASSUMPTIONS AND PROSPECTIVE GAPS ARE PART OF THE MAPPING.

ID	Obligation	Pre-execution verification	Runtime evidence	Cross-stage interpretation
L1	Complete measurements before energy aggregation	NuXmv: aggregation implies three group completions	Unique candidate/group records; DejaVu past-group facts	Ordering agreement when required evidence is present
L2	Candidate validation before optimizer commitment	NuXmv: selection/commit requires three ready candidates	Three validation events precede commit; consumed scores	Agreement of ordering; online enforcement is fail-stop
L3	Current evaluation and parameter provenance	NuXmv: epoch equality and readiness reset	Run/update/evaluation identities, indices, angles, successors	Runtime checks strengthen the epoch abstraction
L4	Terminal closure and declared stopping reason	NuXmv: absorbing terminals and budget bounds only	No later activity; reason checked against oracle/budget	Qualified agreement for shared subclaims; reason is runtime-only
N1	Expectations consistent with raw counts	Dafny normalization kernel and range bound	Exact parity reconstruction and count totals	Agreement under trusted basis and parity mapping
N2	Energy uses declared coefficients and measurements	Dafny contribution, weighted-sum, and interval kernels	Rational reference versus contributions, total energy, actual optimizer input	Arithmetic agreement within frozen tolerance
N3	Physical shots counted consistently	Dafny natural-number list accumulation	Unique group executions, allocations, cumulative totals	Agreement under group-identity assumptions; changed shots invalidate the stochastic setup
N4	Valid candidates and deterministic selection	Dafny modular wrap and integer argmin/tie priority	Recomputed candidate indices, scores, chosen successor	Agreement under trusted angle/gate interpretation
P1	Bounded success probability	Storm reachability of the Gaussian DTMC	First-success frequencies and Wilson intervals	Statistical compatibility under matched conditions
P2	Updates conditional on bounded success	Python propagation of the same Gaussian kernel	Mean first-success update and bootstrap interval	Supporting, multiplicity-adjusted comparison
P3	Shots conditional on bounded success	Fixed shot allocation times P2	Successful-path shots; fixed shots/update	Derived from P2, not independent evidence

TABLE II

FINAL MATCHED CONFIGURATION FROM THE SEALED ASSURANCE RECORD. AN UPDATE CONTAINS THREE CANDIDATE EVALUATIONS AND NINE GROUP EXECUTIONS.

Item	Declared value
Parameter grid	$z \in \{0, \ldots, 16-1\}^4$; 65,536 points
Initial index	$(3, 11, 6, 14)$
Coordinate cycle	$(0, 1, 2, 3)$
Measurement groups	$\{ZI, ZZ\}$, $\{YZ\}$, $\{XX\}$
Shots/group/candidate	1,024
Physical shots/update	9,216
Update bound	$K = 50$
Maximum shots/run	460,800
Success	$E_{\text{ideal}}(z) - E_{\text{ref}} \leq 0.02$
Baseline executions	400; separate seeded streams
Environment	Ideal Born probabilities; finite-shot multinomial counts
Numeric tolerance	10^{-12} for energy and expectations
Gaussian integration	24-node Gauss–Hermite quadrature
Intervals	95% Wilson; 5,000 bootstrap resamples

The DTMC state is the parameter-grid point and next coordinate phase. Success states are absorbing. Storm [5] checks

$$p_G(k) = \Pr_G(T \leq k) \quad \text{using } \mathrm{P}_{=?}[\mathrm{F}^{\leq k} \text{ success}]. \tag{8}$$

This is exhaustive model checking of the constructed Gaussian DTMC, not the physical finite-shot process. Python forward propagation of the same kernel produces the full cumulative curve and first-success distribution. P2 is $\mathbb{E}_G[T \mid T \leq K]$; P3 multiplies P2 by the fixed shots per update. These conditional quantities are not Storm reward-query outputs and exclude failed paths. Total expenditure is reported separately. No certified approximation-error bound follows from quadrature row normalization or agreement between two computations of the same model.

D. Runtime Verification

The canonical executor emits preparation and update context, raw group counts, candidate evaluations, numeric-valid events, optimizer commits, and terminal closure. Python checks each emitted event synchronously and stops the run when a problem appears. It checks current identities, complete measurements, arithmetic, consumed scores, successor parameters, resource totals, and terminal reasons. The implementation records a commit and assigns the successor before checking acceptance; its protection is emit-then-check fail-stop, not a separate pre-authorization protocol.

Saved traces are replayed with the same Python checker class. Online/replay agreement supports deterministic reconstruction of those verdicts, not independent implementation correctness. A separately compiled DejaVu monitor [6] batch-replays five temporal projections covering group ordering, validations before commit, energy after update start, validation after energy, and no activity after stop. It is an independent ordering check with narrower coverage, and does not verify

TABLE III
RETAINED REFINED-CONTROLLER EVALUATION. EACH ROW HAS ONE LOCAL TIMING MEASUREMENT; THIS MODEL IS DISTINCT FROM THE FINAL MATCHED CONTROLLER.

Updates	Terms	True	False	Time (s)
3	2	41	0	0.04
5	3	41	0	0.02
10	5	41	0	0.03
20	10	41	0	0.07

TABLE IV
RETAINED NUMERIC TRACES REPLAYED WITH CHECKER VERSION 2.0. PASS DENOTES RECORD CONFORMANCE UNDER THAT CHECKER; RAW COUNTS ARE UNAVAILABLE IN THE ESTIMATOR TRACE.

Trace	Qubits	Terms	Records	Verdict
Two-qubit Estimator	2	4	25	PASS
Heisenberg	14	287	60	PASS

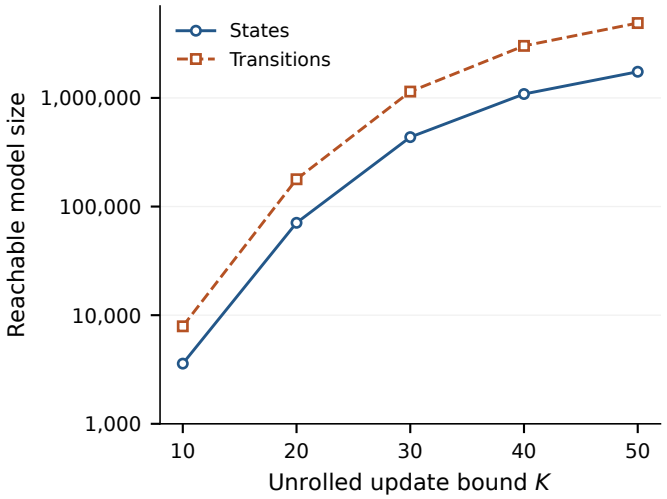


Fig. 2. Reachable states and transitions in the retained time-unrolled Gaussian DTMC study, at the declared grid and shot allocation. The vertical axis is logarithmic; each marker is an archived model size. This representation expands the reachable update-depth history and differs from the fixed-size homogeneous phase DTMC used in the final matched experiment.

arithmetic or authenticate the source of counts. The earlier runtime implementation used an ask/tell acknowledgment protocol; its separate evidence is evaluated below.

IV. INDIVIDUAL LAYER EVALUATION

A. Evidence Selection and Experimental Boundaries

The collective experiment uses exclusively the final run designated in the artifact index and independently audited. Development and earlier primary directories reuse the full-run seeds and are not pooled as independent observations. The retained studies in this section answer layer-specific questions under their own configurations. Their different controllers, stopping rules, measurement strategies, and timing boundaries are not treated as one canonical execution.

The final archive was sealed before its designated runtime execution. Earlier development executions had already exposed the same seeded trajectories; this was an engineering iteration rather than a blinded confirmatory study. The paper's P2/P3 interpretation is a reporting correction after the audit, with primary data and frozen experimental sources preserved. The reproducibility bundle records source hashes and generates numerical text, tables, and plots programmatically.

B. Logical and Numeric Outcomes

The earlier refined controller has 41 true properties in its saved NuXmv output and 18 positive phase-reachability queries in the separate archive. Its budget-abort naming differs from the matched controller's normal budget closure. Table III reports its bounded configurations and single wall-clock measurements. These small timings are not evidence of a general scalability law or of cost as quantum-state dimension grows.

For the canonical matched instance, 7 logical properties hold and the three Dafny verification invocations report 24 verified units with zero errors. These are tool-reported verification units, not a count of independent system-level properties. Their distribution across modules and their correspondence to N1–N4 are retained in the evidence manifest.

The earlier numeric checker also supports an executed Estimator trace and a larger exact-estimate trace, replayed with the later recorded checker in Table IV. The Estimator trace lacks raw measurement counts, so it cannot establish the raw-count reconstruction now available in the matched experiment. The Heisenberg trace illustrates acceptance of a larger term list; it provides no timing-scaling or optimizer-success theorem. Synthetic fixtures are not described as executed molecular instances.

C. Probabilistic Model Size and Cost

The retained scalability study builds a reachable, time-unrolled DTMC for each bound. It merges successful paths into an absorbing success state and horizon-exhausted paths into an absorbing timeout state. Figure 2 shows its recorded states and transitions. At the largest bound, it contains 1,741,676 states and 4,889,851 transitions. This is a different representation from the final homogeneous grid/coordinate-phase DTMC, whose size is fixed at 262,144 states and 755,176 transitions. Increasing a reachability-query horizon on the latter does not create the time-unrolled growth shown in the figure.

Table V separates construction and checking. Construction excludes Gaussian-kernel preparation; checking is the median of the recorded tool-reported repetitions and excludes parsing. A displayed zero is below reporting resolution. The archive retains per-depth state counts, timings, the CSV, metadata, and generation source, but not the exported explicit scalability models. Accordingly, this supports the recorded model-growth and cost measurements, with a weaker raw-model audit trail than the final retained DTMC.

TABLE V
RETAINED TIME-UNROLLED MODEL COST. CONSTRUCTION IS ONE MEASUREMENT; CHECKING IS THE MEDIAN OF THREE TOOL-REPORTED TIMINGS.

K	Construction (s)	Checking (s)
10	0.008	0.000
20	0.161	0.002
30	1.067	0.020
40	2.821	0.072
50	4.663	0.206

TABLE VI
TIMING MEDIANS AND ADDED COST IN SECONDS. THE INTERVAL APPLIES TO THE DIFFERENCE OF INDEPENDENT CONDITION MEDIANS.

Evals.	Baseline	Verified	Added	Added 95% CI
10	0.044	0.630	0.586	[0.579, 0.592]
20	0.087	0.959	0.872	[0.863, 0.891]
30	0.130	1.268	1.139	[1.119, 1.156]
40	0.174	1.562	1.388	[1.366, 1.406]
50	0.215	1.876	1.661	[1.633, 1.721]

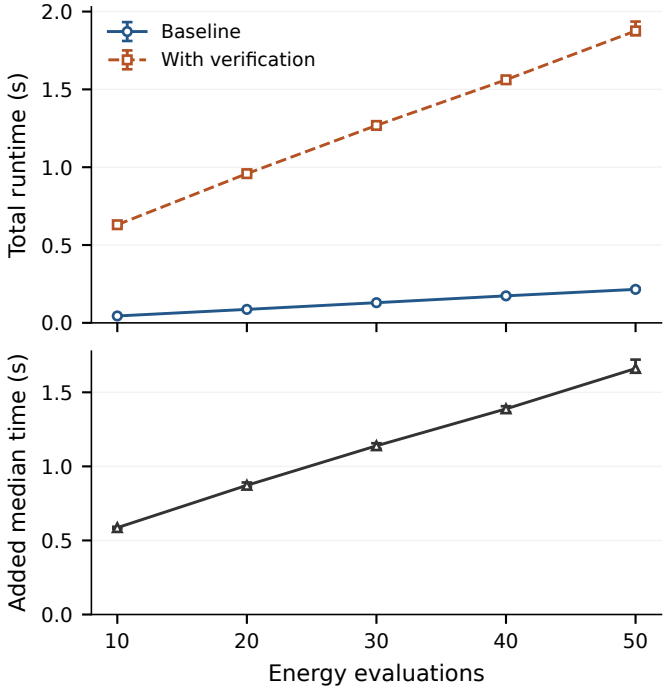


Fig. 3. Retained simulator runtime cost under the separate ask/tell runtime implementation. Top: baseline and verified medians. Bottom: verified median minus baseline median. Error bars are 95% percentile bootstrap intervals from 10,000 independent-condition resamples, using 30 measured runs per condition/count after 5 warm-ups. Timing includes setup, online monitoring, and finalization; fidelity analysis and offline replay are excluded. Repetitions use the same workload seed.

TABLE VII
SEPARATE RUNTIME SIMULATOR AND HARDWARE EVIDENCE, VERIFIED AGAINST ARCHIVED HASHES AND REPORTS. "PASS/PASS" DENOTES ONLINE/REPLAY CONFORMANCE; BOTH USE BUDGET TERMINATION.

Execution	Evals.	PUBs	Shots	Online/replay
statevector_sampler	50	200	204,800	pass/pass
ibm_marrakesh	50	200	204,800	pass/pass

D. Runtime Cost and Hardware Evidence

The completed September runtime cost study has 50 warm-ups and 300 measured runs. Each condition and evaluation count has 5 warm-ups followed by 30 measured repetitions. It uses a StatevectorSampler, separate measurement submissions per Pauli term, an ask/tell pattern-search optimizer, and the recorded stability policy. Reusing the workload seed holds the sampled trajectory fixed while repetitions measure timing variation. This is separate from the final cyclic-grid experiment.

Figure 3 and Table VI report total runtime and the difference of condition medians. The verified timing includes setup, monitor startup, online checking/acknowledgments, and finalization; archived fidelity computation, artifact writing outside the declared boundary, and offline replay are excluded. Bootstrap samples independently resample each condition's measured times. The resulting intervals describe uncertainty in timing summaries for this workload, not variation across independent optimization trajectories.

At the largest tested count, the baseline median is 0.215 s, the verified median is 1.876 s, and the added median time is 1.661 s (771.84 % relative increase). These measurements show substantial relative overhead on a fast simulator; they do not establish the final canonical experiment's overhead or larger-workload scaling. The final matched archive has no repeated uninstrumented timing condition, and numeric verification has no systematic cost study.

The retained runtime archive also contains separately configured sampled simulator and `ibm_marrakesh` executions. Table VII reports their verified measurements and online/replay outcomes. Both close at the evaluation budget rather than certify ideal-energy success. Their acceptance supports observed event, arithmetic, resource, and evidence conformance. A hardware result with backend noise and a different optimizer is not a matched validation of the Gaussian search model.

V. CROSS-LAYER RELATIONSHIPS AND AGREEMENT

A. Experiment Goals and Comparison Procedure

The collective experiment asks whether the frozen prospective subclaims remain consistent with concrete evidence when circuit, operator, parameters, candidate rule, measurement groups, shot allocation, success predicate, and bound are matched. It also tests how controlled faults affect the contract vector. It does not test chemical validity, prove physical measurements, establish general optimizer convergence, or combine the tools into one formal semantics.

Stage 1 generates and checks the controller, verifies the numeric modules, constructs the Gaussian kernel and DTMC, runs Storm at declared checkpoints, compiles the temporal monitor, and seals the assurance record. Stage 2 executes

the identical recorded configuration, stores raw counts and all consumed optimizer values, and performs online and replay checking. The comparison validates the seal and trace hashes and reports each contract separately. A numerical tolerance is part of deterministic conformance; confidence intervals express statistical uncertainty and have a different role.

The implementation labels observed satisfaction `AGREEMENT`, violations `DISAGREEMENT`, invalid assumptions `ASSUMPTION_MISMATCH`, and missing or unavailable checks `INCONCLUSIVE`. We interpret a disagreement according to the contract and evidence. A probabilistic deviation can concern the Gaussian model only after its matched-assumption boundary is established. An individual failed stochastic search is not a violation of P1. The software gates execution on successful Stage 1 results, so it does not evaluate the complete matrix of failed prospective checks against runtime outcomes.

For P1, the final bound is the designated primary comparison. We report pointwise Wilson intervals for all declared checkpoints. For the informative supporting comparisons, the audit-recommended analysis computes two-sided score-test p-values for the nonzero checkpoint probabilities and a normal-approximation test for the successful-run update mean, then applies Holm adjustment to this family. The zero-probability early checkpoints are structurally uninformative; P3 duplicates P2 and is not an additional test. This post-audit adjustment corrects interpretation without replacing the observations or claiming a preregistered equivalence margin.

B. Logical and Numeric Contract Agreement

Table VIII gives the prospective subclaim and runtime count for every logical and numeric contract. All observed baseline obligations agree on 400 runs, subject to the mapping qualifications. This is observed conformance, not proof of all Python executions. In particular, L4 agreement covers the shared terminal-absorption and bound conditions; the stronger termination-reason check has runtime evidence alone.

The raw-count reconstruction covers 40,554 candidate evaluations and 121,662 group executions. The maximum observed energy reconstruction error is 5.542×10^{-17}, below the declared tolerance; expectation reconstruction has zero observed error. Total physical consumption is 124,581,888 shots across 13,518 optimizer updates. Python online and replay verdicts agree for every run. All baseline runs also pass the separate DejaVu temporal replay. These counts are evidence volumes, not numbers of independent proved properties.

C. Gaussian Prediction and Matched Finite-Shot Evidence

Figure 4 compares the full Gaussian first-success curve to empirical execution using exactly the same success predicate. The curve is forward propagation of the retained kernel; Storm independently checks the declared checkpoint probabilities. Their maximum recorded numerical difference is 1.110×10^{-16}. The maximum quadrature row-normalization correction is 5.090×10^{-8}; neither quantity measures error relative to the true finite-shot process.

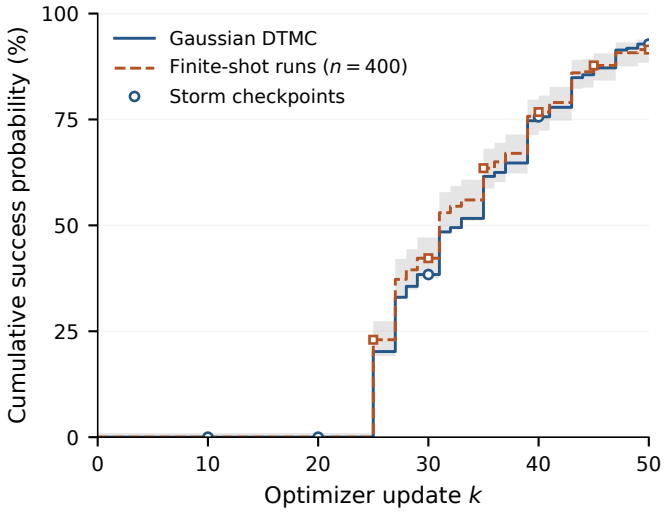


Fig. 4. Cumulative probability of first satisfying $E_{\mathrm{ideal}}(z) - E_{\mathrm{ref}} \leq 0.02$ versus optimizer update. The solid staircase is Gaussian-DTMC forward propagation, circles are Storm checkpoints, and the dashed staircase is the matched ideal-Born finite-shot experiment with 400 independent seeded runs and 1,024 shots per group/candidate. The shaded region gives marginal 95% Wilson intervals at each update; it is not a simultaneous confidence band. No backend noise is included.

At the primary bound, 366/400 runs succeed and 34 terminate normally at budget. The empirical probability is 91.50 %, with Wilson interval [88.36 %, 93.85 %], compared with 92.80 % modeled probability. The absolute difference is 1.30 percentage points. Table IX gives exact checkpoint counts, intervals, deviations, and the supporting multiplicity-adjusted comparisons. Agreement denotes statistical compatibility at the tested resolution, not equality of distributions.

P2 is 33.111 modeled updates conditional on success versus 32.290 empirical updates, with the original percentile-bootstrap interval [31.579, 32.995]. Table X preserves that interval, which does not contain the model mean. The earlier automated report therefore labeled P2 and P3 disagreements using marginal interval containment. The audit identified this as a statistical false alarm: the recomputed raw P2 p-value is 0.024, becoming 0.094 after adjustment. No informative comparison rejects at the family level. The corrected classifications are agreement through statistical compatibility for P2 and the derived P3; the original archived verdict remains unchanged for provenance. A non-rejection supplies neither an equivalence proof nor evidence that the model lies inside every marginal interval.

Including unsuccessful runs, mean expenditure is 33.795 updates versus 34.327 modeled updates. Total shot consumption above includes budget-terminated paths; it must not be confused with the cost conditioned on success. The primary probability comparison's approximate normal-theory deviation detectable at 80% power is 3.62 percentage points at the sample size used here. This is a local planning approximation, not an equivalence margin or a statement that smaller deviations have been excluded. Supporting checkpoints are correlated,

TABLE VIII

FINAL CONTRACT-WISE LOGICAL/NUMERIC RESULTS. THE LISTED PROSPECTIVE SUBCLAIMS HAVE SUCCESSFUL TOOL OUTPUTS; THE RUNTIME COLUMN COUNTS AGREEING BASELINE RUNS. AGREEMENT IS INTERPRETED UNDER TABLE I'S ASSUMPTIONS. L4 DOES NOT CARRY A PROSPECTIVE PROOF OF TERMINATION REASONS.

ID	Prospectively established scope	Runtime agreement	Cross-stage classification
L1	Group-completeness counter	400/400	Agreement under mapping
L2	Validation-readiness counter	400/400	Agreement under mapping
L3	Epoch and readiness reset	400/400	Agreement under mapping
L4	Terminal absorption; budget bounds	400/400	Qualified agreement
N1	Normalization kernel; parity assumed	400/400	Agreement under mapping
N2	Real weighted sum and bound	400/400	Agreement under mapping
N3	List sum; group identity assumed	400/400	Agreement under mapping
N4	Wrap and integer tie-priority selection	400/400	Agreement under mapping

TABLE IX

FINAL MATCHED P1 RESULTS. PROBABILITIES AND WILSON LIMITS ARE PERCENTAGES; DEVIATIONS ARE PERCENTAGE POINTS. HOLM VALUES ADJUST THE INFORMATIVE CHECKPOINT AND P2 COMPARISONS JOINTLY. THE EARLY ZERO-PROBABILITY CHECKPOINTS ARE STRUCTURAL, NOT INDEPENDENT EMPIRICAL CONFIRMATION.

k	Successes/runs	Model (%)	Empirical (%)	Wilson 95% (%)	$\lvert\Delta p\rvert$ (pp)	Holm p
10	0/400	0.00	0.00	[0.00, 0.95]	0.00	–
20	0/400	0.00	0.00	[0.00, 0.95]	0.00	–
30	169/400	38.39	42.25	[37.51, 47.14]	3.86	0.338
40	307/400	75.63	76.75	[72.37, 80.62]	1.12	0.627
50	366/400	92.80	91.50	[88.36, 93.85]	1.30	0.627

TABLE X

CONDITIONAL SUCCESSFUL-PATH RESOURCES IN THE FINAL MATCHED EXPERIMENT, BASED ON 366 SUCCESSFUL EXECUTIONS. INTERVALS ARE THE ORIGINAL 95% PERCENTILE-BOOTSTRAP INTERVALS FROM 5,000 RESAMPLES. P2 IS STATISTICALLY COMPATIBLE AFTER HOLM ADJUSTMENT; P3 SCALES P2 BY 9,216 SHOTS/UPDATE AND IS NOT AN INDEPENDENT TEST.

Contract / unit	Model mean	Empirical mean	Bootstrap 95%	Absolute deviation	Holm p
P2: updates	33.111	32.290	[31.579, 32.995]	0.822	0.094
P3: shots	305,152.7	297,581.1	[291,034.2, 304,077.6]	7,571.6	derived

and the confidence band is pointwise.

The earlier FakeJakarta density-matrix study has 347/400 successes (86.75 %). Its backend noise and compiled circuits fall outside the ideal Gaussian assumptions. We retain it as an out-of-model robustness observation, separate from the matched curve; its gap from the Gaussian prediction is not an estimate of Gaussian approximation error. Independent large-sample audit simulations support the auditor's false-alarm conclusion, but their saved arrays are absent from this workspace. We therefore do not plot them or reproduce their numerical summaries as newly verified paper results.

D. Faults and Cross-Stage Discrepancies

Table XI reports all deterministic runtime verdicts, rather than only the expected target contract. There are 5 fault scenarios, each repeated 10 times, for 50 tests. Execution faults use the first update/candidate boundary; they do not represent diverse injection sites. The missing-evidence scenario deletes counts from a saved trace and retains the corresponding control execution. Thus it assesses evidence handling rather than an execution fault.

Premature commitment is caught by Python and DejaVu ordering checks. Altering the energy or coefficient pairing violates N2 while temporal projections pass, illustrating the different scopes. A shot-allocation change produces an N3 assumption mismatch and makes the probabilistic comparison inapplicable. Its fault-record applicability label is configured by the injector; it should not be advertised as an independently inferred stochastic verdict.

Missing counts make numeric reconstruction inconclusive. The current checker also records an L1 disagreement because its completeness condition conflates unavailable evidence with observed missing measurements. We retain the full vector and qualify this behavior: a deleted record alone does not establish that the original execution aggregated too early. The final data therefore support all expected target detections, but do not support flawless discrepancy classification or broad fault coverage. Terminal-boundary, stale-input, and additional injection-location coverage remain outside this suite.

E. Answers to the Research Questions

The contract matrix and frozen record answer RQ1 through explicit, inspectable mappings. For RQ2, logical model check-

TABLE XI
FULL FINAL FAULT VERDICT VECTORS. A: AGREEMENT; D: DISAGREEMENT; M: ASSUMPTION_MISMATCH; I: INCONCLUSIVE. "TARGET" COUNTS THE EXPECTED TARGET-CONTRACT VERDICT; "DV PASS" IS THE INDEPENDENT TEMPORAL REPLAY COUNT. EACH ROW IS ONE SCENARIO REPEATED WITH DIFFERENT COUNT STREAMS. MISSING EVIDENCE IS A SAVED-TRACE MUTATION. ITS L1 D IS AN AUDITED EVIDENCE-CLASSIFICATION ARTIFACT, NOT PROOF OF A PHYSICAL ORDERING VIOLATION.

Fault	Target	L1	L2	L3	L4	N1	N2	N3	N4	DV pass
coefficient swap	10/10	A	A	A	A	A	D	A	A	10/10
corrupt energy	10/10	A	A	A	A	A	D	A	A	10/10
missing evidence	10/10	D	I	A	A	I	I	I	I	10/10
premature update	10/10	I	D	A	A	I	I	A	A	0/10
shot mismatch	10/10	A	A	A	A	A	I	M	A	10/10

ing, mathematical numeric proofs, Gaussian reachability, runtime conformance, and empirical estimates each establish a different result with stated assumptions. For RQ3, the matched baseline supports logical/numeric agreement for the prospectively checked subclaims and probabilistic compatibility after the corrected statistical interpretation. L4's prospective gap prevents a stronger termination-reason claim. For RQ4, the fault vectors and separate cost measurements quantify the tested coverage and expense, without comparing tools on incompatible workloads or claiming complete fault detection.

VI. RELATED WORK

VQE and variational hybrid algorithms motivate the repeated interaction between classical decisions and quantum measurements [1], [2]. Their energy-estimation objective supplies the numeric obligation, while uncertainty and bounded search motivate the probabilistic one. NuXmv, Dafny, Storm, and DejaVu provide established model-checking, deductive-verification, probabilistic-analysis, and temporal-monitoring mechanisms [3]–[6]. Our contribution concerns their contract traceability and two-stage use in an implemented hybrid workflow; it is not a new proof engine or a formalism identifying their guarantees.

Statistical intervals and multiple-comparison control address a different inference problem from formal verification. Wilson intervals describe a binomial proportion [7]; Holm adjustment controls the family-wise rejection rate for a specified family of tests [8]. Neither converts observed conformance into exhaustive correctness. The P2 correction illustrates why contract agreement must distinguish a stochastic observation, a marginal test, and the interpretation of related comparisons.

VII. DISCUSSION AND LIMITATIONS

Verification scope. NuXmv is exhaustive for its encoded controller; Dafny proves kernels under mathematical semantics and preconditions; Storm is exhaustive for the constructed Gaussian DTMC. Runtime checking covers observed instrumented traces. Repeated executions provide statistical evidence. There is no theorem proving refinement of all projections to the Python program or composing them into unconditional correctness.

Trusted base. Circuit and measurement-basis generation, bit order and parity interpretation, event instrumentation, proof tools, and the runtime reference checker are trusted. Hashes bind the artifacts to the recorded experiment but do not authenticate that counts originated from a physical backend. Consistent fabrication or relabeling is outside the claimed protection. The normal-CDF approximation and quadrature have no certified global stochastic-error bound.

Generality and observability. The canonical instance is a small two-qubit benchmark with a fixed finite grid, ideal statistics, and an exact-energy stopping oracle. Its ideal target, cyclic optimizer, grouped measurements, and finite budget limit transfer to hardware or other VQE instances. The separate hardware and larger numeric traces demonstrate particular checks, not a matched cross-stage guarantee for those executions.

Statistical and audit boundaries. The P2/P3 correction is disclosed as post-audit. Original marginal intervals and observations remain visible. Sample size constrains resolution, and a non-rejection is not equivalence. Development runs reused seeds and are excluded from sample counts. The independent audit found useful specification gaps, including termination reasons and evidence classification; this draft narrows claims rather than implying those frozen sources were repaired. Missing audit arrays and unretained scalability exports are documented in the reproducibility manifest.

Closed-loop scope. The framework detects inconsistencies and supports revising assumptions or contracts before another sealed execution. The implemented feedback is manual. Its gated workflow does not test all possible combinations of prospective failure and runtime behavior, and its fault scenarios cover a limited boundary.

VIII. CONCLUSION

The two-stage framework connects family-owned VQE contracts to prospective results and concrete runtime evidence through a frozen canonical configuration. The final matched experiment supports contract-wise consistency within the stated logical and numeric subclaims and statistical compatibility of bounded Gaussian predictions and finite-shot observations, including the corrected P2/P3 interpretation. Full fault vectors and separately configured cost studies identify both useful checks and evidence limits. The result is an auditable relationship among heterogeneous verification lay-

ers, with obligations, assumptions, and interpretations kept explicit.

Artifact Availability

The local reproducibility bundle includes this source, vector figures, editable architecture source, figure-generation scripts, generated numerical tables, `FIGURE_PLAN.md`, `FIGURE_MANIFEST.md`, and `PAPER_RESULT_TRACEABILITY.md`. The source manifest records hashes of the final assurance record, models, proof logs, raw traces, and retained layer-specific inputs. Original experimental artifacts are preserved; no public archive or DOI is claimed.

References

[1] A. Peruzzo, J. McClean, P. Shadbolt, M.-H. Yung, X.-Q. Zhou, P. J. Love, A. Aspuru-Guzik, and J. L. O'Brien, "A variational eigenvalue solver on a photonic quantum processor," *Nature Communications*, vol. 5, p. 4213, 2014.

[2] J. R. McClean, J. Romero, R. Babbush, and A. Aspuru-Guzik, "The theory of variational hybrid quantum-classical algorithms," *New Journal of Physics*, vol. 18, no. 2, p. 023023, 2016.

[3] R. Cavada, A. Cimatti, M. Dorigatti, A. Griggio, A. Mariotti, A. Micheli, S. Mover, M. Roveri, and S. Tonetta, "The nuXmv symbolic model checker," in *Computer Aided Verification*. Springer, 2014, pp. 334–342.

[4] The Dafny Project, *Dafny Reference Manual*, version 4.11.0; accessed October 6, 2026. [Online]. Available: https://dafny.org/v4.11.0/DafnyRef/DafnyRef

[5] C. Hensel, S. Junges, J.-P. Katoen, T. Quatmann, and M. Volk, "The probabilistic model checker Storm," *International Journal on Software Tools for Technology Transfer*, vol. 24, no. 4, pp. 589–610, 2022.

[6] K. Havelund, D. Peled, and D. Ulus, "DejaVu: A monitoring tool for first-order temporal logic," 2018. [Online]. Available: https://www.havelund.com/Publications/dejavu-mtcps-2018.pdf

[7] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[8] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979. [Online]. Available: https://www.jstor.org/stable/4615733